

Versioni degli strumenti usati:

BNFC - 2.9.5

GHC - 9.8.2

Happy - 2.0.2

Alex - 3.5.1.0

Il progetto è stato strutturato in moduli distinti, che corrispondono alle diverse fasi della compilazione. Un primo modulo è composto da Lexer e Parser (per costruirli si è fatto uso di BNFC). Le direttive di BNFC specificano un Abstract Syntax Tree polimorfo. Il risultato della fase di lexing e parsing è l'istanziamento di quest'ultimo con le posizioni (riga e colonna) dei token. Questo AST viene trasformato in un AST semplificato (sempre polimorfo), la cui sintassi è stata da noi specificata (modulo `AST.hs`), che viene utilizzato come base per la fase di analisi di semantica statica e per quella successiva di generazione del Three Address Code, ciascuna corrispondente a un modulo separato.

Un programma nel linguaggio implementato non prevede un entrypoint esplicito (es. una funzione `main`), ma è costituito da un blocco, che si compone da una serie di istruzioni, a loro volta `statement` o espressioni.

In qualsiasi punto all'interno dei blocchi è possibile dichiarare costanti, variabili e funzioni. Tutte le dichiarazioni prevedono la contemporanea inizializzazione. Costanti e variabili sono visibili solo dal punto di dichiarazione in poi, mentre per quanto riguarda le dichiarazioni di funzioni abbiamo la `full visibility`.

Le costanti sono gestite come valori determinati al momento della compilazione, possono essere inizializzate con espressioni che comprendono altre costanti anche non precedentemente dichiarate.

Le variabili possono essere inizializzate con espressioni generiche. Variabili e costanti condividono lo stesso namespace, mentre le funzioni/procedure hanno un namespace separato. Le costanti devono essere scritte con caratteri maiuscoli (il caso contrario viene generato un `warning`).

Nella seguente tabella elenchiamo gli operatori usati con la semantica consueta. L'operatore `&` è usato per l'operazione di referenziazione, il `$` per la dereferenziazione.

Operatore	Precedenza	Associatività
<code> </code>	1	Sinistra
<code>&&</code>	2	Sinistra
<code>!</code>	3	
<code>==, !=, <, <=, >, >=</code>	4	
<code>+, -</code> (binario)	5	Sinistra
<code>*, /, %</code>	6	Sinistra
<code>**</code>	7	Destra
<code>-</code> (unario)	8	
<code>++, --</code> (pre e post)	9	
<code>&, \$</code>	10	

Per il passaggio di parametri le modalità ammesse sono per valore e per riferimento.

I casting impliciti sono ammessi solamente verso l'alto (secondo il reticolo dei tipi definito nella seguente sezione) e ogni tipo numerico ha una sintassi letterale associata.

A livello di linguaggio ammettiamo il tipo stringa con l'indicizzazione come unica operazione consentita.

Per la generazione di Three Address Code abbiamo deciso di utilizzare uno stato costruito sulla base di una monade di stato e un "monad transformer" `StateT s (m :: * -> *)` (entrambi forniti dalla libreria `Control.Monad.Trans.State`).

Lo stato comprende i seguenti elementi:

- un intero usato per generare gli identificatori dei temporanei e delle label,
- un intero per stringhe statiche
- una lista di label
- una lista di istruzioni TAC
- una lista di label sospese per un uso successivo (che permette l'uso della tecnica `fall-through` nella generazione di TAC per le espressioni booleane)
- una lista di coppie di label per permettere la gestione delle istruzioni di `break` e `continue` all'interno del costrutto di iterazione indeterminata (`while ... do ...`)
- una mappa con valori stringhe e label identificative

Per gestire la definizione di funzioni innestate lo stato viene salvato e ripristinato grazie all'uso del "monad transformer".

Negli assegnamenti la valutazione dell'l-value precede quella dell-r-value. La valutazione delle espressioni avviene da sinistra a destra, così come quella degli argomenti delle funzioni o procedure. La valutazione delle espressioni booleane avviene ovunque con modalità `short-cut` (guardie, passaggio dei

parametri, assegnamenti).

Type System

The character ; is used a separator between statements.

The definition of the set of types $\mathbb{T}_1$ is defined as follows.

$$\begin{aligned}\mathbb{T}_1 &= \{\mathbf{bool}, \mathbf{char}, \mathbf{int}, \mathbf{float}, \mathbf{string}\} \cup \{\&\tau \mid \tau \in \mathbb{T}_1\} \cup \{[n]\tau \mid n \in \mathbb{N}, \tau \in \mathbb{T}_1\} \\ \mathbb{T}_2 &= \mathbb{T}_1 \cup \{\mathbf{void}\} \cup \{\mu_1\tau_1 \times \dots \times \mu_n\tau_n \mapsto \tau \mid \tau \in \mathbb{T}_1 \cup \mathbf{void}, \forall i(\mu_i \in \{\mathbf{byRef}, \mathbf{byVal}\} \wedge \tau_i \in \mathbb{T}_1)\}\end{aligned}$$

We introduce the **compatibility** relation $\leq \subseteq \mathbb{T}_2 \times \mathbb{T}_2$, where $\tau_1 \leq \tau_2$ denotes that the type τ_1 is compatible with the type τ_2 . Moreover, it holds that:

- $\forall \tau \in \mathbb{T}_2 \quad \tau \leq \tau$.
- $\forall n \in \mathbb{N} \wedge \tau_1, \tau_2 \in \mathbb{T}_2 \quad \tau_1 \leq \tau_2 \longrightarrow \neg([n]\tau_1 \leq [n]\tau_2) \wedge \neg(\&\tau_1 \leq \&\tau_2)$.

Notice that the partially ordered set $(\mathbb{T}_2, \leq)$ has the least upper bound property. We denote it with $\sqcup$ defined as follows:

$$\tau_1 \sqcup \tau_2 = \begin{cases} \tau_1 & \tau_1 = \tau_2 \\ \mathbf{int} & (\tau_1 = \mathbf{int} \wedge \tau_2 = \mathbf{char}) \vee (\tau_1 = \mathbf{char} \wedge \tau_2 = \mathbf{int}) \\ \mathbf{float} & (\tau_1 = \mathbf{float} \wedge \tau_2 = \mathbf{char}) \vee (\tau_1 = \mathbf{char} \wedge \tau_2 = \mathbf{float}) \\ \mathbf{float} & (\tau_1 = \mathbf{int} \wedge \tau_2 = \mathbf{float}) \vee (\tau_1 = \mathbf{float} \wedge \tau_2 = \mathbf{int}) \\ \mathbf{error} & \text{otherwise} \end{cases}$$

Variable declarations (V)

variable declaration

$$\frac{env \vdash_E e : (\tau_2, r_v) \quad \tau_1 \geq \tau_2}{env_{ext}, env \vdash_V |\mathbf{var} \ id : \tau_1 := e| : \{(id, \tau_1)\}}$$

sequences

$$\begin{aligned} & \frac{env_2 \vdash_S stmt \quad env_{ext}, env_1 \vdash_V v^{\text{decl}} : env_3 \quad env_2 = env_{ext} \dot{\cup} env_1}{env_{ext}, env_1 \vdash_V |stmt ; v^{\text{decl}}| : env_3} \\ & \frac{env_{ext}, env_1 \vdash_V v^{\text{decl}} : env_2 \quad env_3 \vdash_S stmt \quad env_3 = env_{ext} \dot{\cup} (env_1 \dot{\cup} env_2)}{env_{ext}, env_1 \vdash_V |v^{\text{decl}} ; stmt| : env_2} \\ & \frac{env_{ext}, env_1 \vdash_V v_1^{\text{decl}} : env_2 \quad env_{ext}, env_3 \vdash_V v_2^{\text{decl}} : env_4 \quad env_3 = env_1 \dot{\cup} env_2}{env_{ext}, env_1 \vdash_V |v_1^{\text{decl}} ; v_2^{\text{decl}}| : env_2 \dot{\cup} env_4} \\ & \frac{env_2 \vdash_S stmt \quad env_2 = env_{ext} \dot{\cup} env_1}{env_{ext}, env_1 \vdash_V stmt : \emptyset} \end{aligned}$$

Function declarations (F)

We define $\text{fonly}(env) = \{(id, \dots \mapsto \tau) : (id, \dots \mapsto \tau) \in env\}$, such that $\text{fonly}(env)$ contains only function definitions. This lets us avoid full visibility for variable declarations.

function declaration

$$\frac{\tau = m_1^p \tau_1^p \times \dots \mapsto \tau^f \quad env_2, env_3 \vdash_F f_{body} : env_x \quad env_2 = env_{ext} \dot{\cup} env_1 \quad env_3 = \{(id^f, \tau), (\mathbf{return}, \tau^f), (id_1^p, \tau_1^p), \dots\}}{env_{ext}, env_1 \vdash_F |\mathbf{def} \ id^f (m_1^p id_1^p : \tau_1^p, \dots) : \tau^f \{f_{body}\}| : \{(id^f, \tau)\}}$$

sequences

$$\begin{aligned} & \frac{env_{ext}, env_5 \vdash_V v^{\text{decl}} : env_2 \quad env_{ext}, env_3 \vdash_F f^{\text{decl}} : env_4 \quad env_3 = env_1 \dot{\cup} env_2 \quad env_5 = env_1 \dot{\cup} \text{fonly}(env_4)}{env_{ext}, env_1 \vdash_F |v^{\text{decl}} ; f^{\text{decl}}| : env_2 \dot{\cup} env_4} \\ & \frac{env_{ext}, env_1 \vdash_F f^{\text{decl}} : env_2 \quad env_{ext}, env_3 \vdash_V v^{\text{decl}} : env_4 \quad env_3 = env_1 \dot{\cup} env_2}{env_{ext}, env_1 \vdash_F |f^{\text{decl}} ; v^{\text{decl}}| : env_2 \dot{\cup} env_4} \\ & \frac{env_{ext}, env_4 \vdash_F f_1^{\text{decl}} : env_2 \quad env_{ext}, env_3 \vdash_F f_2^{\text{decl}} : env_4 \quad env_3 = env_1 \dot{\cup} env_2 \quad env_5 = env_1 \dot{\cup} \text{fonly}(env_4)}{env_{ext}, env_1 \vdash_F |f_1^{\text{decl}} ; f_2^{\text{decl}}| : env_2 \dot{\cup} env_4} \\ & \frac{env_{ext}, env_1 \vdash_V v^{\text{decl}} : env_2}{env_{ext}, env_1 \vdash_F v^{\text{decl}} : env_2} \end{aligned}$$

Expressions

$$\frac{env(id) = \tau \quad \tau \in \mathbb{T}_2}{env \vdash_E id : (\tau, l_v)}$$

indexed accessor

$$\frac{env \vdash_E e_1 : ([n]\tau_1, \omega) \quad env \vdash_E e_2 : (\tau_2, r_v) \quad \tau_2 \leq \text{int}, n \in \mathbb{N}, \omega \in \{l_v, r_v\}}{env \vdash_E |e_1[e_2]| : (\tau_1, \omega)}$$

$$\frac{env \vdash_E e_1 : (\text{string}, l_v) \quad env \vdash_E e_2 : (\tau, r_v) \quad \tau \leq \text{int}}{env \vdash_E |e_1[e_2]| : (\text{char}, r_v)}$$

literals

$$\frac{Type = \begin{cases} \text{bool} \mapsto \text{bool} \\ \text{char} \mapsto \text{char} \\ \text{integer} \mapsto \text{int} \\ \text{float} \mapsto \text{float} \\ \text{string} \mapsto \text{string} \end{cases}}{env \vdash_E \text{literal} : (Type(\text{literal}), r_v)}$$

array literal

$$\frac{\exists \tau \forall i (env \vdash_E e_i : (\tau_i, r_v) \wedge \tau_i \leq \tau) \quad \nexists i_1, i_2 (env \vdash_E e_{i_1} : (\tau_1, r_v) \wedge env \vdash_E e_{i_2} : (\tau_2, r_v) \wedge \tau_1 \neq \tau_2)}{env \vdash_E [e_1 \dots e_n] : ([n]\tau, r_v)}$$

binary operators

$$\frac{\text{binaryMathOp} \in \{+, -, *, /, **\} \quad env \vdash_E e_1 : (\tau_1, r_v) \quad env \vdash_E e_2 : (\tau_2, r_v) \quad (\tau_1 \sqcup \tau_2) \leq \text{float}}{env \vdash_E |e_1 \text{ binaryOp } e_2| : (\tau_1 \sqcup \tau_2, r_v)}$$

$$\frac{env \vdash_E e_1 : (\tau_1, r_v) \quad env \vdash_E e_2 : (\tau_2, r_v) \quad (\tau_1 \sqcup \tau_2) \leq \text{int}}{env \vdash_E |e_1 \% e_2| : (\tau_1 \sqcup \tau_2, r_v)}$$

$$\frac{\text{binaryOp} \in \{||, \&\&\} \quad env \vdash_E e_1 : (\text{bool}, r_v) \quad env \vdash_E e_2 : (\text{bool}, r_v)}{env \vdash_E |e_1 \text{ binaryOp } e_2| : (\text{bool}, r_v)}$$

unary operators

$$\frac{\text{unariMathOp} \in \{++, --\} \quad env \vdash_E e : (\tau, l_v) \quad \tau \leq \text{float}}{env \vdash_E \text{unariMathOp } e : (\tau, r_v)}$$

$$\frac{\text{unariMathOp} \in \{++, --\} \quad env \vdash_E e : (\tau, l_v) \quad \tau \leq \text{float}}{env \vdash_E e \text{ unariMathOp} : (\tau, r_v)}$$

$$\frac{env \vdash_E e : (\text{bool}, r_v)}{env \vdash_E !e : (\text{bool}, r_v)}$$

$$\frac{env \vdash_E e : (\tau, r_v) \quad \tau \leq \text{float}}{env \vdash_E -e : (\tau \sqcup \text{int}, r_v)}$$

$$\frac{env \vdash_E e : (\tau, l_v)}{env \vdash_E \&e : (\&\tau, r_v)}$$

$$\frac{env \vdash_E e : (\&\tau, l_v)}{env \vdash_E \$e : (\tau, l_v)}$$

comparators

$$\frac{\text{comparatorOp} \in \{=, !=, <, <=, >, >=\} \quad env \vdash_E e_1 : (\tau_1, r_v) \quad env \vdash_E e_2 : (\tau_2, r_v) \quad (\tau_1 \sqcup \tau_2) \leq \text{float}}{env \vdash_E |e_1 \text{ comparatorOp } e_2| : (\text{bool}, r_v)}$$

function call

$$\frac{env(id) = \mu_1 \tau_1 \times \dots \times \mu_n \tau_n \mapsto \tau \quad \forall i ((\mu_i = \text{byRef} \implies env \vdash_E e_i : (\tau'_i, l_v) \wedge \tau'_i \sqcup \tau_i = \tau_i) \wedge (\mu_i = \text{byVal} \implies env \vdash_E e_i : (\tau_i, r_v)) \wedge \tau'_i \sqcup \tau_i)}{env \vdash_E id(e_1 \dots e_n) : (\tau, r_v)}$$

value

$$\frac{env \vdash_E e : (\tau, l_v)}{env \vdash_E e : (\tau, r_v)}$$

Statements (S)

block

$$\frac{env_1, \emptyset \vdash_F f^{\text{decl}} : env_x}{env \vdash_S \{f^{\text{decl}}\}}$$

assignment

$$\frac{env \vdash_E e_1 : (\tau_1, l_v) \quad env \vdash_E e_2 : (\tau_2, r_v) \quad \tau_1 \geq \tau_2}{env \vdash_S e_1 := e_2}$$

$$\frac{o \in \{+, -, *, /, **, \&\&, ||\} \quad env \vdash_E e_1 : (\tau_1, l_v) \quad env \vdash_E e_2 : (\tau_2, r_v) \quad \tau_1 \geq \tau_2}{env \vdash_S e_1 \circ = e_2}$$

branch

$$\frac{env \vdash_E e : (bool, r_v) \quad env \vdash_S b}{env \vdash_S if \ e \ b}$$

$$\frac{env \vdash_E e : (bool, r_v) \quad env \vdash_S b_1 \quad env \vdash_S b_2}{env \vdash_S if \ e \ b_1 \ b_2}$$

$$\frac{env \vdash_E e : (bool, r_v) \quad env \vdash_S b}{env \vdash_S while \ e \ b}$$

jump

$$\frac{env(break) = void}{env \vdash_S break}$$

$$\frac{env(continue) = void}{env \vdash_S continue}$$

$$\frac{env(return) = \tau \quad env \vdash_E e : (\tau', r_v) \quad \tau' \leq \tau}{env \vdash_S return \ e}$$

$$\frac{env(return) = void}{env \vdash_S return}$$